

COS341 2026 - Semester Practical, Phase 1 (Front-End) User Manual - Group 28

Group Members:

- Brayden Butler - 24824713
- Nathan Chisadza - 24825532
- Finnley Wyllie - 24754120
- Daniel Cohen - 24772756

1. What This Program Does

This program is a lexer + SLR(1) parser for the Students' Programming Language (SPL), as defined in the COS341 2026 syntax specification.

Given a plain ASCII text file containing an SPL program, the program either:

- writes a structured "tree.xml" file containing the program's syntax tree (if the input is syntactically valid SPL), or
- prints a syntax or lexical error message to the console, with a hint about what went wrong (if the input is not valid SPL).

2. How to Run the Program

The submitted executable is named:

```
group-28.jar
```

To run it, open a command prompt / terminal in the folder containing the executable, and run:

```
java -jar group-28.jar examples/SPL.txt tree.xml
```

Where: examples/SPL.txt is the path to the plain-text SPL source file to be parsed (e.g. SPL.txt). tree.xml is the path where the syntax tree should be written if parsing succeeds (e.g. tree.xml).

Example:

```
group-28.jar SPL.txt tree.xml
```

3. Expected Behaviour

On SUCCESS (valid SPL program):

- The program prints a short confirmation message to the console.
- A file is written at the given output path (e.g. tree.xml) containing the syntax tree, with:
 - a root node for the SPL_PROG start symbol,
 - inner nodes for every non-terminal in the derivation,
 - leaf nodes for every terminal token consumed from the input, each with a unique ID, its contents, and (for non-root nodes) a reference to its parent's ID; and (for root/inner nodes) a list of its children's IDs.
- The program exits with status code 0.

On FAILURE (invalid SPL program):

- No output file is written.
- A lexical or syntax error message is printed to the console (stderr), including the line and column where the problem was detected, and a short hint about what was expected.
- The program exits with a non-zero status code.

4. Notes on the SPL Language

- Every token in an SPL program must be terminated by a blank space (space, or newline/return).
 - User-defined names (variables and function names) must begin with '#', e.g. #x, #counter, #add1.
 - The end-of-file marker "\$" referenced in the grammar is a theoretical symbol only. It must never be typed in an input file, and it will not appear in the generated tree.xml output.
-

5. System Requirements

- No installation required; the executable is self-contained and can be run directly.